EXPERIMENT – 03

Aim: Implement following optimization algorithms –

1. Momentum based GD
2. Nesterov Accelerator GD
3. Adagrad
4. RMSProp
5. Adamgrad

Note: For implementing above algorithms any of the following training methods can be used:

a. Vanilla Gradient Descent
b. Stochastic Gradient Descent
c. Mini Batch Gradient Descent

Theory:

Gradient descent is a fundamental optimization algorithm in machine learning used to minimize functions by iteratively moving towards the minimum. It's important for training models by fine-tuning parameters to reduce prediction errors.

Momentum-Based Gradient Descent

Momentum-Based Gradient Descent is an enhancement of the standard gradient descent algorithm that aims to accelerate convergence, particularly in the presence of high curvature, small but consistent gradients, or noisy gradients. It introduces a velocity term that accumulates the gradient of the loss function over time, thereby smoothing the path taken by the parameters. The update rule for Momentum-Based Gradient Descent is:

$$v_t = \gamma v_{t-1} + \eta \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_t$$

where:

- v_t is the velocity at iteration t,
- γ is the momentum term (typically between 0 and 1),
- $\eta$is the learning rate,
- $\nabla J(\theta_t)$ is the gradient of the loss function $J(\theta)$ with respect to θ at iteration t.

Advantages of Momentum-Based Gradient Descent

- Accelerated Convergence: Helps in faster convergence, especially in scenarios with small but consistent gradients.
- Smoother Updates: Reduces the oscillations in the gradient updates, leading to a smoother and more stable convergence path.
- Effective in Ravines: Particularly effective in dealing with ravines or regions of steep curvature, common in deep learning loss landscapes.

Disadvantages of Momentum-Based Gradient Descent

- Additional Hyperparameter: Introduces an additional hyperparameter (momentum term) that needs to be tuned.
- Complex Implementation: Slightly more complex to implement compared to standard gradient descent.
- Potential Overcorrection: If not properly tuned, the momentum can lead to overcorrection and instability in the updates.

Nesterov-Based Gradient Descent

Nesterov Accelerated Gradient is a refined version of momentum-based optimization, designed to further smooth and speed up convergence. It does so by looking ahead at the future gradient and adjusting accordingly, which helps reduce overshooting and improves stability.

How NAG Works:

1. Momentum Update: Traditional momentum calculates an update based on the current gradient and a weighted sum of past gradients, creating a “velocity” term that guides the update direction.
2. NAG Update: Instead of calculating the gradient at the current point, NAG takes a look-ahead step using the momentum term. It then computes the gradient at this new look-ahead position and adjusts the update based on this gradient. This approach makes it easier to predict when the optimizer is likely to overshoot, allowing for more controlled updates.

Advantages of NAG

- Reduced Oscillations: By peeking ahead, NAG reduces oscillations around the minimum, which can be especially useful in error surfaces with multiple peaks and valleys.
- Faster Convergence: NAG often converges faster than simple momentum-based methods because it reduces unnecessary steps around the optimum.

Adagrad Gradient Descent

Adagrad stands for Adaptive Gradient Optimizer. There were optimizers like Gradient Descent, Stochastic Gradient Descent, mini-batch SGD, all were used to reduce the loss function with respect to the weights. The weight updating formula is as follows:

$$(\mathrm{w})_{\mathrm{new}} = (\mathrm{w})_{\mathrm{old}} - \eta \frac{\partial \mathrm{L}}{\mathrm{dw}(\ \mathrm{old}\ )}$$

Based on iterations, this formula can be written as:

$$w_t = w_{t-1} - \eta \frac{\partial L}{\partial w(t-1)}$$

where

w(t) = value of w at current iteration, w(t-1) = value of w at previous iteration and η = learning rate.

In SGD and mini-batch SGD, the value of η used to be the same for each weight, or say for each parameter. Typically, η = 0.01. But in Adagrad Optimizer the core idea is that each weight has a different learning rate (η). This modification has great importance, in the real-world dataset, some features are sparse (for example, in Bag of Words most of the features are zero so it's sparse) and some are dense (most of the features will be noon-zero), so keeping the same value of learning rate for all the weights is not good for optimization. The weight updating formula for adagrad looks like:

$$\mathrm{w_t} = \mathrm{w_{t-1}} - \eta'_{\mathrm{t}} \frac{\partial \mathrm{L}}{\partial \mathrm{w(t-1)}}$$

Where alpha(t) denotes different learning rates for each weight at each iteration.

$$\eta'_{\mathrm{t}} = \frac{\eta}{\mathrm{sqrt}(\alpha_{\mathrm{t}} + \epsilon)}$$

Here, η is a constant number, epsilon is a small positive value number to avoid divide by zero error if in case alpha(t) becomes 0 because if alpha(t) become zero then the learning rate will become zero which in turn after multiplying by derivative will make w(old) = w(new), and this will lead to small convergence.

Advantages of Adagrad:

- No manual tuning of the learning rate required.
- Faster convergence
- More reliable

One **disadvantage** of Adagrad optimizer is that alpha(t) can become large as the number of iterations will increase and due to this \eta_t^' will decrease at the larger rate. This will make the old weight almost equal to the new weight which may lead to slow convergence.

RMSProp Gradient Descent

RMSProp (Root Mean Squared Propagation) is an adaptive learning rate optimization algorithm. It is an extension of the popular Adaptive Gradient Algorithm and is designed to dramatically reduce the amount of computational effort used in training neural networks. This algorithm works by exponentially decaying the learning rate every time the squared gradient is less than a certain threshold. This helps reduce the learning rate more quickly when the gradients become small. In this way, RMSProp is able to smoothly adjust the learning rate for each of the parameters in the network, providing a better performance than regular Gradient Descent alone.

The RMSprop algorithm utilizes exponentially weighted moving averages of squared gradients to update the parameters. Here is the mathematical equation for RMSprop:

1. Initialize parameters:
    - Learning rate: α
    - Exponential decay rate for averaging: γ
    - Small constant for numerical stability: ε
    - Initial parameter values: θ
2. Initialize accumulated gradients (Exponentially weighted average):
    - Accumulated squared gradient for each parameter: Et= 0
3. Repeat until convergence or maximum iterations:
    - Compute the gradient of the objective function with respect to the parameters: $g_t = \nabla_\theta J(\theta_t)$
    - Update the exponentially weighted average of the squared gradients: $E_t = \gamma E_{t-1} + (1-\gamma) g_t^2$
    - Update the parameters: $\theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{E_t + \epsilon}}$

where,

- g_t is the gradient of the loss function with respect to the parameters at time t
- γ is a decay factor
- E_t is the exponentially weighted average of the squared gradients
- α is the learning rate
- ϵ is a small constant to prevent division by zero

Adam Gradient Descent

Adaptive Moment Estimation is an algorithm for optimization technique for gradient descent. The method is really efficient when working with large problem involving a lot of data or parameters. It requires less memory and is efficient. Intuitively, it is a combination of the 'gradient descent with momentum' algorithm and the 'RMSP' algorithm.

Working

Adam optimizer involves a combination of two gradient descent methodologies:

Momentum:
This algorithm is used to accelerate the gradient descent algorithm by taking into consideration the 'exponentially weighted average' of the gradients. Using averages makes the algorithm converge towards the minima in a faster pace.

$$w_{t+1} = w_t - \alpha m_t$$

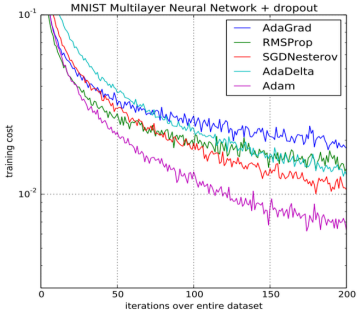
where,

$$m_t = \beta m_{t-1} + (1 - \beta) \left[\frac{\delta L}{\delta w_t} \right]$$

mt = aggregate of gradients at time t [current] (initially, mt = 0)
mt-1 = aggregate of gradients at time t-1 [previous]
Wt = weights at time t
Wt+1 = weights at time t+1
αt = learning rate at time t
∂L = derivative of Loss Function
∂Wt = derivative of weights at time t
β = Moving average parameter (const, 0.9)

Performance:

Building upon the strengths of previous models, Adam optimizer gives much higher performance than the previously used and outperforms them by a big margin into giving an optimized gradient descent. The plot is shown below clearly depicts how Adam Optimizer outperforms the rest of the optimizer by a considerable margin in terms of training cost (low) and performance (high).



Batch Gradient Descent	Stochastic Gradient Descent
Computes gradient using the whole Training sample	Computes gradient using a single Training sample
Slow and computationally expensive algorithm	Faster and less computationally expensive than Batch GD
Not suggested for huge training samples.	Can be used for large training samples.
Deterministic in nature.	Stochastic in nature.
Gives optimal solution given sufficient time to converge.	Gives good solution but not optimal.
No random shuffling of points are required.	The data sample should be in a random order, and this is why we want to shuffle the training set for every epoch.
Can't escape shallow local minima easily.	SGD can escape shallow local minima more easily.
Convergence is slow.	Reaches the convergence much faster.
It updates the model parameters only after processing the entire training set.	It updates the parameters after each individual data point.
The learning rate is fixed and cannot be changed during training.	The learning rate can be adjusted dynamically.
It typically converges to the global minimum for convex loss functions.	It may converge to a local minimum or saddle point.
It may suffer from overfitting if the model is too complex for the dataset.	It can help reduce overfitting by updating the model parameters more frequently.

Comparison between the variants of Gradient Descent

Variant	Data Used	Convergence	Memory Usage	Efficiency	Key Advantage
Batch Gradient Descent	Entire dataset	Stable but slow	High (entire dataset)	Computationally expensive	Stable convergence, global view of data
Stochastic Gradient Descent	One example per iteration	Fast but noisy	Low (one example)	Less efficient	Faster convergence, good for online learning
Mini-Batch Gradient Descent	Mini-batch of data	Faster and smoother	Medium (mini-batch)	Efficient, parallelizable	Balance of speed and stability
Momentum-Based Gradient Descent	Entire dataset or mini-batch	Faster and smoother	Medium (like Mini-Batch)	Efficient with momentum	Accelerated convergence, smooth updates